

Backend

Módulo: Core

Application

Papel: inicializar a aplicação e montar os principais objetos do sistema.

Atributos:

- `gameLoop : GameLoop`
- `appController : AppController`

Métodos:

- `start()` → inicia a aplicação
 - `initialize()` → instância os módulos principais
-

GameLoop

Papel: manter o loop principal do programa rodando.

Atributos:

- `running : boolean` (se o ciclo estiver acontecendo, true, senão, false)
- `targetFPS : int` (quantos quadros por segundo queremos que o loop processe, ou seja, a frequência de atualização da interface)
- `timeSystem : TimeSystem`
- `appController : AppController`

Métodos:

- `run()` → executa o loop principal
- `stop()` → encerra o loop
- `processFrame()` → processa um único quadro do loop
- `update(deltaRealTime)` → atualiza o sistema com base no tempo real
- `render()` → chama a renderização da interface

TimeSystem

Papel: controlar o tempo da simulação e a aceleração temporal.

Atributos:

- `simulationTime : double`
- `timeScale : double`
- `deltaSimulationTime : double`
- `timetarget : double`

Métodos:

- `update(deltaRealTime)` → converte tempo real em tempo simulado
- `setTimeScale(scale)` → altera a velocidade da simulação (na verdade, permite alterar a variável `timetarget`, uma vez que a `timescale` deverá ser calculada a partir dele.
- `getSimulationTime()` → retorna o tempo acumulado da simulação
- `getDeltaSimulationTime()` → retorna o delta do último frame
- `reset()` → reinicia o tempo simulado

AppController

Papel: fazer a ponte entre frontend e backend.

Atributos:

- `simulator : Simulator`
- `mainWindow : MainWindow`

Métodos:

- `processInput()` → recebe ações da UI
- `update(deltaSimulationTime)` → atualiza a simulação
- `render()` → envia dados atualizados para a interface
- `startSimulation()` → manda o simulador iniciar
- `resetSimulation()` → reinicia o ciclo
- `applyConfiguration(configData)` → aplica configurações vindas da UI
- `runStressTest()` → inicia teste de estresse
- `runOverloadTest()` → inicia teste de sobrecarga

Módulo: Simulator

Simulator

Papel: orquestrar toda a simulação.

Atributos:

- `simulationTime : double`
- `cycleNumber : int`
- `vehiclesServed : int`
- `totalEnergyDelivered : double`
- `fleetManager : FleetManager`
- `stationManager : StationManager`
- `eventQueue : EventQueue`
- `eventLogger : EventLogger`
- `simulationConfig : SimulationConfig`
- `currentReport : SimulationReport`
- `cycleActive : boolean`

Métodos:

- `initializeSimulation()` → prepara o ambiente inicial
- `startCycle()` → inicia um novo ciclo
- `update(deltaSimulationTime)` → atualiza a simulação
- `processEvent(event)` → trata um evento retirado da fila
- `scheduleEvent(event)` → adiciona evento na fila
- `endCycle()` → encerra o ciclo atual
- `generateReport()` → cria o relatório do ciclo
- `reset()` → reinicia o simulador
- `getSimulationState()` → retorna dados para a UI

Event

Papel: representa um evento isolado da simulação.

Atributos:

- `timestamp : double`

- `type : EventType`
- `vehicle : Vehicle`
- `chargingPointId : int`

Métodos:

- `getTimestamp()` → retorna o tempo do evento
 - `getType()` → retorna o tipo
 - `getVehicle()` → retorna o veículo associado
 - `getChargingPointId()` → retorna o ponto associado
-

EventQueue

Papel: armazenar e ordenar eventos por prioridade temporal.

Atributos:

- `events : PriorityQueue<Event>`

Métodos:

- `pushEvent(event)` → adiciona um evento
 - `popNextEvent()` → retorna e remove o próximo evento
 - `peekNextEvent()` → retorna o próximo sem remover
 - `isEmpty()` → verifica se a fila está vazia
 - `clear()` → limpa todos os eventos
-

EventLogger

Papel: registrar o histórico textual da simulação.

Atributos:

- `eventLog : List<String>`

Métodos:

- `log(message)` → adiciona mensagem ao log
- `getLog()` → retorna o log atual
- `clearLog()` → limpa o log
- `exportLog()` → exporta o histórico

SimulationReport

Papel: reunir os dados finais de cada ciclo.

Atributos:

- `cycleNumber : int`
- `vehiclesServed : int`
- `totalEnergyDelivered : double`
- `totalRevenue : double`
- `simulationDuration : double`
- `vehicleSummaries : List<String>`

Métodos:

- `generate()` → monta os dados do relatório
 - `addVehicleSummary(summary)` → adiciona resumo de veículo
 - `exportToFile()` → exporta relatório
 - `reset()` → limpa os dados internos
-

EventType (enum)

Papel: definir os tipos possíveis de eventos.

Valores:

- `VEHICLE_ARRIVAL`
 - `START_CHARGING`
 - `FINISH_CHARGING`
 - `POWER_REBALANCE`
 - `SIMULATION_END`
-

Módulo: Fleet Manager

VehicleModel

Papel: representa a configuração de fábrica de um modelo de veículo.

Atributos:

- name : String (Compacto, Sedan, SUV, Caminhonete)
- batteryCapacity : double
- maxChargingPower : double
- idBase : int (100, 200, 300, 400)

Métodos:

- getName()
 - getBatteryCapacity()
 - getMaxChargingPower()
 - getIdBase()
-

Vehicle

Papel: representar um veículo individual na simulação.

Atributos:

- id : int
- model : VehicleModel
- color : String
- batteryCapacity : double
- currentEnergy : double (gerado randomicamente)
- maxChargingPower : double
- arrivalTime : double
- chargingStartTime : double
- chargingFinishTime : double
- waitingTime : double
- priorityScore : double
- status : VehicleStatus
- connectedPoint : ChargingPoint

Métodos:

- getStateOfCharge() → retorna o percentual de bateria
- getEnergyNeeded() → retorna a energia restante
- startCharging(currentTime) → marca início de carga
- stopCharging() → encerra carregamento
- updateEnergy(deltaEnergy) → soma energia à bateria
- updateWaitingTime(currentTime) → calcula espera

- `setPriorityScore(score)` → atualiza prioridade
 - `getPriorityScore()` → retorna prioridade
 - `isFullyCharged()` → verifica se chegou a 100%
-

VehicleStatus (enum)

Papel: indicar o estado do veículo.

Valores:

- `WAITING`
 - `ARRIVING`
 - `CHARGING`
 - `FINISHED`
-

FleetManager

Papel: gerar aleatoriamente os veículos de cada ciclo.

Atributos:

- `availableModels : List<VehicleModel>`
- `colorOptions : List<String>`
- `maxVehiclesPerCycle : int`
- `randomGenerator`
- `modelCounters : int`

Métodos:

- `generateVehicles()` → gera os veículos do ciclo
 - `createRandomVehicle()` → cria um único veículo aleatório
 - `generateArrivalDelay()` → gera atraso de chegada
 - `generateInitialBattery()` → gera bateria inicial aleatória
 - `generateRandomColor()` → escolhe cor aleatória
 - `generateRandomModel()` → escolhe modelo aleatório
 - `generateVehicleCount()` → decide quantos veículos serão gerados
-

Módulo: Station Manager

ChargingPoint

Papel: representar um totem/poste de carregamento.

Atributos:

- `id : int`
- `isOccupied : boolean`
- `connectedVehicle : Vehicle`
- `currentPower : double`
- `energyDelivered : double`

Métodos:

- `isAvailable()` → verificar disponibilidade
 - `connectVehicle(vehicle)` → conecta um veículo
 - `deliverEnergy(deltaSimulationTime)` → entrega energia ao veículo
 - `setCurrentPower(power)` → define potência alocada
 - `getCurrentPower()` → retorna potência atual
 - `getConnectedVehicle()` → retorna o veículo conectado
 - `reset()` → limpa o estado do totem
-

SmartQueue

Papel: gerenciar a fila inteligente de espera.

Atributos:

- `queue : PriorityQueue<Vehicle>`
- `weightBattery : double`
- `weightWaitingTime : double`

Métodos:

- `addVehicle(vehicle)` → adiciona veículo à fila
 - `getNextVehicle()` → remove e retorna o mais prioritário
 - `calculatePriority(vehicle)` → calcula score
 - `updatePriorities(currentTime)` → recalcula prioridades
 - `isEmpty()` → verifica se está vazia
 - `clear()` → limpa a fila
-

EnergyManager

Papel: distribuir potência entre os veículos conectados.

Atributos:

- `stationMaxPower : double` (definido nas configurações)
- `activePoints : List<ChargingPoint>` (depende do número de pontos conectados a cada ciclo)

Métodos:

- `rebalancePower()` → executa o balanceamento
 - `calculateTotalDemand()` → soma demandas dos veículos ativos
 - `getActivePoints()` → retorna os pontos ativos
 - `distributePower()` → distribui potência
 - `distributeRemainingPower()` → redistribui sobra
 - `applyPriorityDistribution()` → distribuição com prioridade, se usada
-

StationManager

Papel: gerenciar os totens, a fila e o processo de carga.

Atributos:

- `chargingPoints : List<ChargingPoint>`
- `waitingQueue : SmartQueue`
- `energyManager : EnergyManager`
- `energyPrice : double`

Métodos:

- `addVehicleToQueue(vehicle)` → envia veículo para fila
 - `assignVehiclesToPoints()` → conecta veículos por ordem/prioridade
 - `updateCharging(deltaSimulationTime)` → atualiza carga em todos os pontos
 - `getActiveVehicles()` → retorna veículos em carga
 - `calculateChargingCost(vehicle)` → calcula custo em reais
 - `getAvailablePoint()` → encontrar um totem livre
 - `reset()` → limpa a estação
-

Módulo: Config

SimulationConfig

Papel: armazenar as configurações globais da simulação.

Atributos:

- `stationMaxPower` : double
- `energyPrice` : double
- `defaultTimeScale` : double
- `maxVehiclesPerCycle` : int
- `stressTestEnabled` : boolean

Opcionais:

- `priorityWeightBattery` : double
- `priorityWeightWaiting` : double
- `vehicleModelConfigs` : List<VehicleModel>

Métodos:

- `loadDefault()` → carrega valores padrão
 - `setStationPower(power)` → permite personalizar a potência disponível
 - `setEnergyPrice(price)` → permite personalizar o valor da energia
 - `setTimeScale(scale)` → permite alterar a escala de tempo do carregamento
 - `setMaxVehiclesPerCycle(max)` → permite alterar a quantidade máxima de veículos por ciclo
 - `enableStressTest(enabled)` → automaticamente gera o máximo de veículos no ciclo
 - `setPriorityWeights(batteryWeight, waitingWeight)` → permite alterar atributos do algoritmo de balanceamento e prioridade alterando os pesos do nível de bateria ou ordem de chegada (quanto tempo o carro está esperando)
 - `updateVehicleModel(model)` → altera configuração de um modelo
-

Frontend

Módulo: Window

MainWindow

Papel: janela principal da aplicação.

Atributos:

- `stationView : StationView`
- `controlPanel : ControlPanel`
- `eventLogPanel : EventLogPanel`
- `statsPanel : StatsPanel`
- `configWindow : ConfigWindow`

Métodos:

- `draw()` → desenha a janela principal
 - `update()` → atualiza os elementos da UI
 - `handleInput()` → encaminha inputs
 - `openConfigWindow()` → exibe popup de configuração
 - `closeConfigWindow()` → fecha popup
-

ConfigWindow

Papel: popup/modal de configuração.

Atributos:

- `stationPowerSlider : Slider`
- `energyPriceSlider : Slider`
- `batteryPrioritySlider : Slider`
- `waitingPrioritySlider : Slider`
- `timeScaleSlider : Slider`
- `applyButton : Button`
- `closeButton : Button`
- `isVisible : boolean`

Métodos:

- `open()` → torna visível
- `close()` → oculta
- `draw()` → renderiza a janela
- `handleInput()` → trata eventos do usuário
- `collectConfigData()` → coleta os valores preenchidos

Módulo: Panels

ControlPanel

Papel: exibir botões principais de controle da simulação.

Atributos:

- `startButton : Button`
- `resetButton : Button`
- `stressTestButton : Button`
- `overloadTestButton : Button`
- `speedSlider : Slider`

Métodos:

- `draw()`
 - `handleInput()`
 - `bindActions()` → associa ações aos botões
-

EventLogPanel

Papel: exibir os eventos textuais da simulação.

Atributos:

- `logEntries : List<String>`
- `titleLabel : Label`

Métodos:

- `draw()`
 - `updateLog(entries)` → recebe e mostra os logs
 - `clear()`
-

StatsPanel

Papel: exibir estatísticas gerais em tempo real.

Atributos:

- `vehiclesServedLabel : Label`
- `totalEnergyLabel : Label`
- `simulationTimeLabel : Label`
- `stationPowerLabel : Label`

Métodos:

- `draw()`
 - `updateStats(simulationState)` → atualiza as métricas na interface
-

Módulo: Views

StationView

Papel: desenhar a estação e organizar os elementos visuais dos 8 pontos.

Atributos:

- `chargerViews : List<ChargerView>`
- `vehicleViews : List<VehicleView>`

Métodos:

- `drawStation()`
 - `updateVehicles(vehicles)` → atualiza carros exibidos
 - `updateChargers(points)` → atualiza estado dos totens
-

VehicleView

Papel: representar visualmente um veículo na interface.

Atributos:

- `vehicleID : int`
- `sprite`
- `batteryBar : ProgressBar`
- `modelLabel : Label`
- `colorLabel : Label`

Métodos:

- `draw()`
 - `updateBattery(soc)` → atualiza barra de bateria
 - `updateSprite(imagePath)` → altera imagem do carro
 - `setPosition(x, y)` → define posição visual
-

ChargerView

Papel: representar visualmente um poste/totem de carregamento.

Atributos:

- `chargerID : int`
- `occupied : boolean`
- `connectedVehicleView : VehicleView`
- `powerLabel : Label`
- `statusLabel : Label`

Métodos:

- `draw()`
 - `updateState(pointState)` → atualiza estado visual
 - `attachVehicle(vehicleView)` → associa um carro visualmente
 - `detachVehicle()` → remove o carro do ponto
-

Módulo: Components

UIComponent

Papel: classe base para componentes reutilizáveis.

Atributos:

- `x : double`
- `y : double`
- `width : double`
- `height : double`
- `visible : boolean`

Métodos:

- `draw()`
 - `handleInput()`
 - `setVisible(visible)`
 - `setPosition(x, y)`
-

Button

Papel: botão clicável.

Atributos:

- `text : String`
- `onClick`
- `enabled : boolean`

Métodos:

- `draw()`
 - `handleInput()`
 - `setOnClick(action)`
 - `setEnabled(enabled)`
-

Label

Papel: mostrar texto estático ou dinâmico.

Atributos:

- `text : String`

Métodos:

- `draw()`
 - `setText(text)`
 - `getText()`
-

ProgressBar

Papel: mostrar progresso visual, como bateria.

Atributos:

- `value : double`
- `maxValue : double`

Métodos:

- `draw()`
 - `setValue(value)`
 - `setMaxValue(maxValue)`
 - `getPercentage()`
-

Slider

Papel: permitir escolha contínua de valores.

Atributos:

- `minValue : double`
- `maxValue : double`
- `currentValue : double`

Métodos:

- `draw()`
 - `handleInput()`
 - `setValue(value)`
 - `getValue()`
-

Módulo: Controller

UIController

Papel: controlar a comunicação da interface com o simulador.

Atributos:

- `simulator : Simulator`

Métodos:

- `startSimulation()`
- `pauseSimulation()`
- `resetSimulation()`
- `openConfig()`
- `applySettings(configData)`
- `runStressTest()`
- `runOverloadTest()`
- `getSimulationState()` → retorna dados para views e panels
- `getEventLog()` → retorna o log para a UI